Week 4: Advanced Threat Detection & Web Security Enhancements

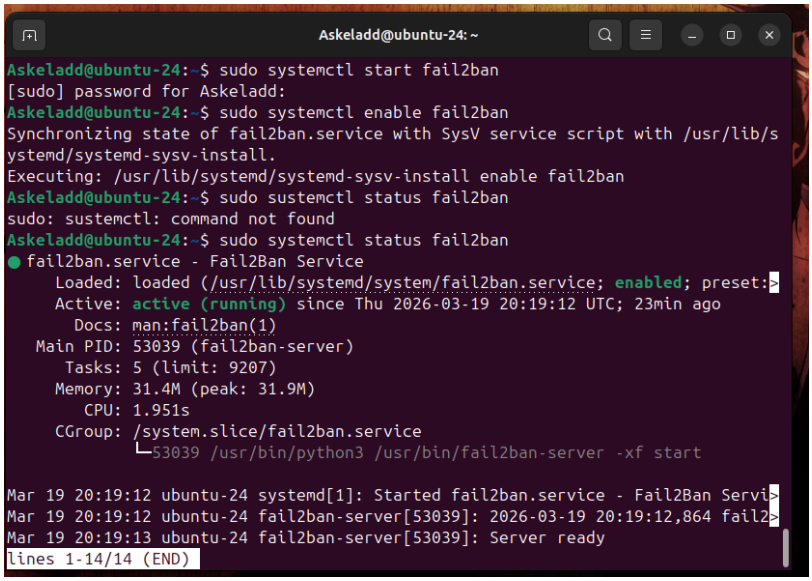
Objective

The objective of this week was to implement advanced security mechanisms to protect a web application from common threats, detect suspicious activities in real-time, and secure API endpoints using industry best practices.

1. Intrusion Detection & Monitoring

To detect and prevent unauthorized access attempts, an intrusion detection system was implemented using **Fail2Ban** on an Ubuntu Virtual Machine.

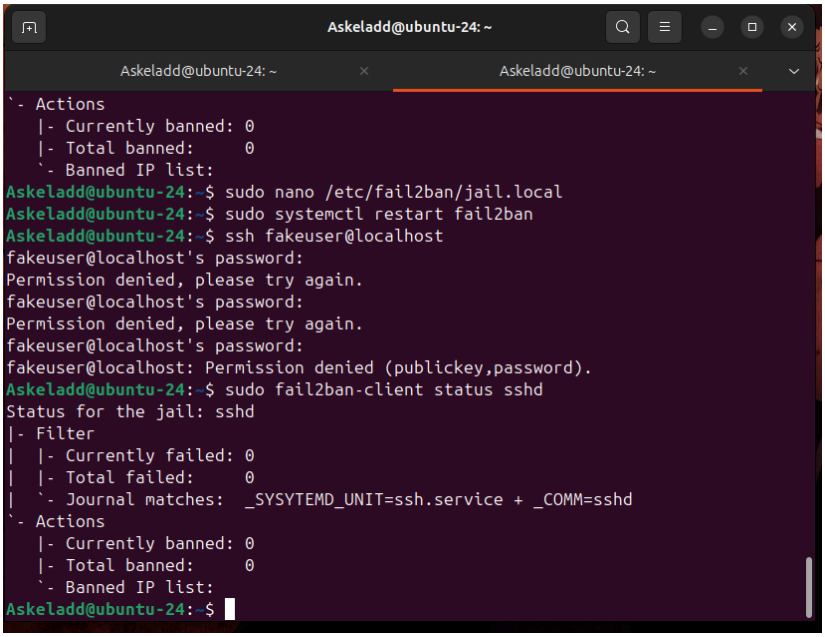
Fail2Ban was configured to monitor SSH login attempts and identify repeated failed login activities. A custom configuration was applied where an IP address is automatically banned after **three failed login attempts within a defined time window**.



Show: `sudo systemctl status fail2ban` **(active running)**

To validate the system, multiple failed login attempts were simulated using incorrect credentials. After exceeding the allowed number of retries, Fail2Ban successfully detected the suspicious behavior and banned the attacking IP address.

Show: `jail.local` file with sshd settings (maxretry, bantime)



Show: `sudo fail2ban-client status sshd`

Output should include: **Banned IP list**

This confirms that the intrusion detection mechanism is functioning correctly and can mitigate brute-force attacks in real time.

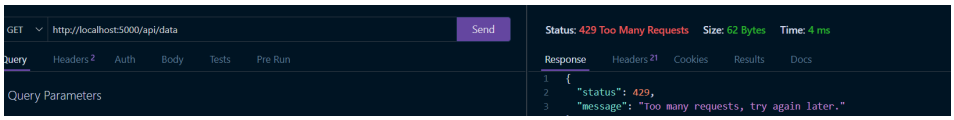
---

2. API Security Hardening

The backend API was secured using multiple layers of protection to prevent unauthorized access and abuse.

Rate Limiting

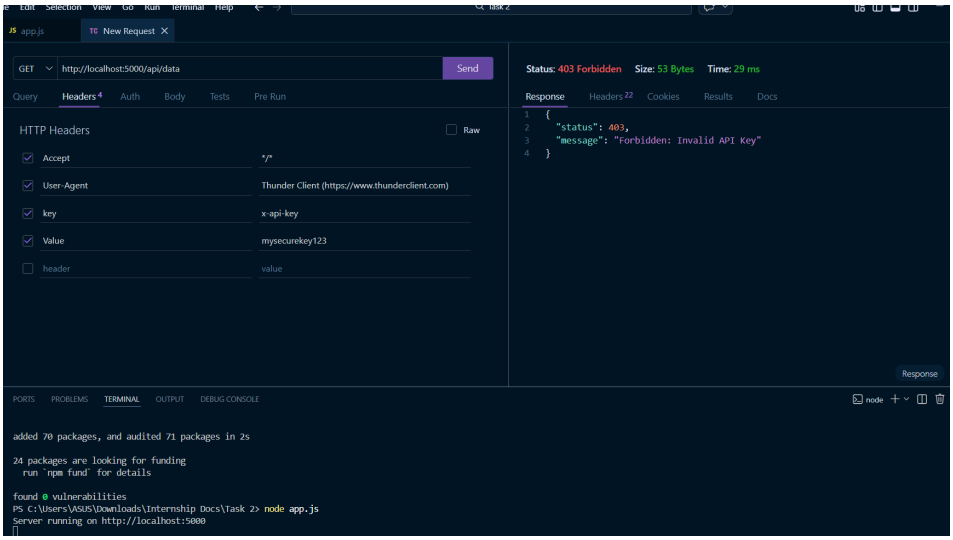
Rate limiting was implemented to restrict the number of requests a client can send within a specific time frame. This helps prevent brute-force attacks and API abuse. When the request limit is exceeded, the server responds with a **429 (Too Many Requests)** error.



Show Postman response with **429 error**

API Key Authentication

An API key-based authentication system was implemented to ensure that only authorized users can access the API. Requests without a valid API key are rejected with a **403 (Forbidden)** response.



Show Postman response with **403 Forbidden**

CORS Configuration

CORS was configured to allow requests only from trusted origins. This prevents unauthorized domains from accessing the backend API.

```
/* ==============================
   🌐 3. CORS CONFIGURATION
============================== */
app.use(
  cors({
    origin: "http://localhost:3000", // allow only your frontend
    methods: ["GET", "POST"],
  })
);
```

Show request blocked or mention restricted origin in code

These combined mechanisms ensure that the API is protected from unauthorized access, excessive requests, and malicious interactions.

---

3. Security Headers & Content Security Policy (CSP)

To further enhance web security, HTTP security headers were implemented using middleware.

Content Security Policy (CSP)

A strict Content Security Policy was applied to control which resources can be loaded by the browser. This helps prevent **Cross-Site Scripting (XSS)** attacks.

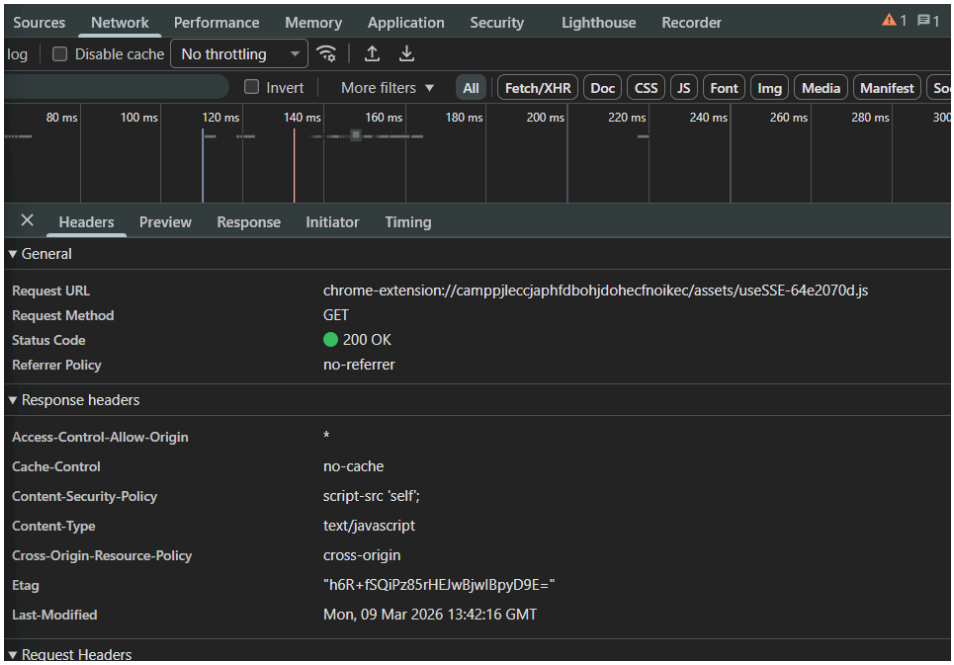
HTTP Strict Transport Security (HSTS)

HSTS was enabled to enforce secure HTTPS communication, ensuring encrypted data transfer.

Additional Security Headers

Additional headers were implemented to:

- Prevent clickjacking
- Disable MIME sniffing
- Hide server details



Open browser → Inspect → Network → Headers

Show:

- Content-Security-Policy
- Strict-Transport-Security
- X-Frame-Options

These headers significantly improve the overall security of the web application.

Results & Observations

- Fail2Ban successfully detected and blocked malicious login attempts
- Rate limiting prevented excessive API requests
- API authentication restricted unauthorized users
- CORS blocked untrusted domains
- Security headers improved browser-level protection